What is STLC

STLC is a structured process that defines the **step-by-step activities involved in software testing** from requirement analysis to test closure.

It ensures testing is **planned, systematic, and measurable**, rather than random execution.

Phases of STLC

1. Requirement Analysis

What happens: Understand what needs to be tested from requirements or user stories.

Example: For a login system, QA identifies requirements like valid login, invalid login, password rules, etc.

2. Test Planning

What happens: Define strategy, scope, resources, and timeline.

Example: Decide to use Selenium for automation and Postman for API testing, and set regression scope.

3. Test Case Design (Test Development)

What happens: Write test cases and prepare test data.

Example:

- Valid login test case
- Invalid password test case

4. Test Environment Setup

What happens: Prepare the system where testing will be executed.

Example: QA environment deployed with database, APIs, and frontend connected.

5. Test Execution

What happens: Run test cases and record results.

Example: Execute login test → compare expected vs actual result.

6. Defect Reporting & Tracking

What happens: Log bugs and track fixes.

Example: Report “Login fails for valid user” in Jira and assign to developer.

7. Test Closure

What happens: Evaluate testing completion and prepare summary report.

Example: Final report shows 95% test cases passed, 5% bugs fixed or deferred.

Testing Activities

1. Test Planning

Define **objectives, scope, strategy, resources, schedule, and risks**.

Example: Decide to test an e-commerce app using UI + API automation with Playwright + Postman.

2. Test Monitoring & Control

Track progress against the plan and take corrective actions.

Example: If API testing is delayed, redistribute effort or adjust timeline.

3. Test Analysis (What to test?)

Analyze requirements/test basis and derive **test conditions / acceptance criteria / testable items**, including risks.

Example: From “Login feature”, derive conditions like valid login, invalid credentials, locked account, security checks.

4. Test Design (How to test?)

Convert test conditions into **test cases, test data, test coverage approach, and test design techniques**.

Example:

- Decide boundary value tests for password length
- Decide equivalence partitioning for input validation

- Design data sets and coverage strategy

5. Test Implementation

Prepare execution-ready assets (**testware**).

- Organize test cases into suites
- Create automation scripts/manual procedures
- Set up test environment

Example: Build Playwright framework and configure staging environment.

6. Test Execution

Run tests and compare **actual vs expected results**, log defects, and analyze failures.

Example: API returns 500 instead of 200 → defect logged.

7. Test Closure

Complete testing cycle and finalize reporting.

- Close defects and testware
- Archive artifacts
- Prepare test summary report
- Collect lessons learned

Example: Release report with pass/fail metrics and known issues.

Software Development Lifecycle (SDLC)

The Software Development Life Cycle (SDLC) refers to a methodology with clearly defined processes for creating high-quality software.

SDLC Phases are:

1. Requirement Gathering & Analysis

Collect and analyze business/user requirements.

Output: SRS (Software Requirement Specification)

Example: Define features like login, payment, user roles.

2. System Design (architectural/UI/UX)

Convert requirements into system UI, UX and system architecture and design.

Output: HLD (High-Level Design: System architecture (services + components)), LLD (Low-Level Design: Internal design of a component (API, logic, DB schema))

Example: Decide database schema, API structure, system flow.

3. Implementation (Coding/Development)

Develop the actual software based on design documents.

Output: Source code

Example: Backend APIs in Node.js, frontend in React.

4. Testing

Verify the system meets requirements and is defect-free.

Output: Test reports, bug reports

Example: API testing, UI testing, regression testing.

5. Deployment

Release the software to production or live environment.

Output: Live system

Example: Deploy application on AWS, VPS, or cloud platform.

6. Maintenance

Fix issues and improve the system after release.

Output: Updates, patches, enhancements

Example: Bug fixes, performance improvements, new feature updates.

Types of SDLC Models

1. Sequential Model

In a sequential model, development follows a fixed step-by-step flow, such as:

Requirement → Design → Development → Testing → Deployment

Testers usually get involved in requirement review, test analysis, and test design early. However, because executable code is developed in later phases, **dynamic testing** usually starts late.

Example: Waterfall, V-Model

Simple idea: Testing preparation can start early, but actual execution starts after development.

2. Iterative / Incremental Model

In iterative and incremental models, the system is developed in repeated cycles. Each cycle may deliver a working prototype or product increment.

This allows both:

- **Static testing**: reviewing requirements, design, test cases
- **Dynamic testing**: executing the working software

Because increments are delivered frequently, teams need fast feedback and strong regression testing.

Example: Spiral model, Prototyping, RAD

Simple idea: Build, test, improve, and deliver step by step.

3. Agile Methodology

Agile is a flexible and collaborative SDLC approach where software is developed in short cycles with frequent customer feedback.

It combines both **iterative** and **incremental** development. Cross-functional teams work together to deliver working software quickly and adapt to changing requirements.

Example: Scrum, Kanban

Simple idea: Build small, test continuously, get feedback, and improve fast.

Development Approach: ATDD, TDD and BDD Approach

Acceptance Test-Driven Development (ATDD)

- Derives tests from acceptance criteria as part of the system design process
- Tests are written before the part of the application is developed to satisfy the tests

Test-Driven Development (TDD):

- Directs the coding through test cases (instead of extensive software design)
- Tests are written first, then the code is written to satisfy the tests, and then the tests and code are refactored

Behavior-Driven Development (BDD):

- Expresses the desired behavior of an application with test cases written in a simple form of natural language, which is easy to understand by stakeholders usually using the given/When/Then format.
- Test cases are then automatically translated into executable tests

For all these approaches, automated tests can be kept to maintain code quality during future changes or improvements

For real development practice:

***ATDD-style thinking is more common before development**, where requirements are broken into acceptance criteria.*